

# TuTracker en Next.js

Migrar la app a Next.js + TypeScript, aprendiendo el porqué de cada pieza mientras la construimos — sobre tu propio producto, no un tutorial genérico.

**Stack** Next.js + TypeScript    **Alcance** puerto 1:1, sin rediseño todavía    **Backend** Supabase, sin cambios

00

## Cómo funciona cada lección

No es código que te entrego para aprobar — es código que construimos juntos, con la explicación integrada. Cada lección tiene tres momentos:

### 1 · Concepto

Antes de tocar código, te explico la idea de React/Next.js que entra en juego y por qué existe (no solo "cómo se usa").

### 2 · Código de referencia

Yo porto la primera pieza de ese tipo, con comentarios que explican cada decisión — el "por qué así" junto a cada línea que importa.

### 3 · Práctica guiada

Tú portas la siguiente pieza parecida siguiendo el mismo molde, abres el PR, y lo revisamos juntos antes de mergear.

### Queda un manual, no solo una conversación

Cada lección se guarda como una nota corta en `docs/nextjs-course/` dentro del repo: el concepto, el código comentado, y la decisión tomada. Al terminar tienes tu propio manual de Next.js, escrito sobre tu propia app.

01

# Lo que ya decidimos

- ✓ **Next.js + TypeScript** — mismo stack que Dipzon.Dev, sin pila nueva desde cero.
- ✓ **Puerto 1:1 primero** — misma app, mismo look, ahora en componentes. El rediseño visual va después, sobre esta base.
- ✓ **Supabase intacto** — auth, tablas, RLS, Edge Functions y el cron del push siguen exactamente igual.
- ✓ **CSS actual se conserva tal cual** — los tokens de marca (día/noche) pasan a `globals.css` sin tocarlos; no metemos Tailwind todavía, eso es decisión del rediseño, no de esta migración.

02

## El temario

10 lecciones, en orden — cada una depende de la anterior. Las **DE REFERENCIA** las hago yo explicando; las **DE PRÁCTICA** las haces tú con el mismo molde.

### 0 Por qué Next.js, por qué TypeScript

REFERENCIA

**Concepto:** qué resuelve un framework de componentes que HTML+JS plano no resuelve a esta escala; qué es un sistema de tipos y por qué importa antes de meter pagos.

**No se porta código todavía** — es la lección de contexto.

### 1 El proyecto y el App Router

REFERENCIA

**Concepto:** `create-next-app`, la carpeta `app/`, y la diferencia entre **Server Components** (por defecto) y **Client Components** ("`use client`") — cuál usar y cuándo.

**Se porta:** el esqueleto — `manifest.webmanifest`, iconos, fuentes, `globals.css` con los tokens de marca.

## 2 Tu primer componente — el login

REFERENCIA

**Concepto:** qué es un componente, props, useState, y por qué el login tiene que ser Client Component (necesita interactividad y el SDK de Supabase corriendo en el navegador).

**Se porta:** #gate → <LoginGate />.

## 3 El estado — adiós a la variable global DB

REFERENCIA

**Concepto:** por qué React no "muta y vuelve a pintar a mano" como hacíamos con innerHTML — el estado vive en un store (Zustand, simple de aprender) y la UI se actualiza sola cuando cambia.

**Se porta:** el modelo DB, normalize(), mergeDB() — la lógica no cambia, cambia dónde vive.

## 4 Conectar Supabase — el hook de sesión y sync

REFERENCIA

**Concepto:** qué es un **custom hook** en React — encapsular lógica reusable (aquí: sesión, pull/push, merge) detrás de una función que cualquier componente puede usar.

**Se porta:** initSync(), pushRemote(), pullRemote() → useSync().

## 5 El patrón lista + editor — Hábitos

REFERENCIA

**Concepto:** composición de componentes — cómo una lista, una fila y un modal-editor se combinan y se reutilizan, en vez de repetir renderX() por cada sección como en la versión vieja.

**Se porta:** Hábitos completo (lista + editor). **Este es el molde** para la lección 6.

## 6 Práctica — Rutina, Metas y Fechas

TU TURNO

**Aplicas el molde de la lección 5** a las otras tres secciones que comparten la misma forma (lista + editor). Yo reviso cada PR contigo, línea por línea si hace falta.

**Portas:** Rutina semanal, Metas generales, Fechas importantes.

## 7 Retos compartidos — llamadas a funciones RPC

REFERENCIA

**Concepto:** cómo se ven las llamadas a `supabase.rpc()` (crear/unirse/renombrar/eliminar reto) dentro de un componente, y el patrón de **estado optimista** vs. esperar respuesta del servidor.

**Se porta:** el leaderboard, el flujo de invitación, el CRUD del dueño.

## 8 PWA y push en Next.js

REFERENCIA

**Concepto:** cómo se registra un Service Worker dentro de un framework (ya no es un `<script>` suelto), y **por qué hay que re-probar en tu iPhone** los bugs de isla dinámica y diálogos anidados que ya arreglamos — un pipeline de render nuevo puede reabrirlos.

**Se porta:** `sw.js`, el flujo de instalar app, el recordatorio push.

## 9 Deploy y verificación final

REFERENCIA

**Concepto:** variables de entorno en Vercel con Next.js (vs. las llaves sueltas en el HTML de hoy), y cómo comparar la app nueva contra la vieja función por función antes de apagar la anterior.

**Resultado:** TuTracker corriendo en Next.js, mismo comportamiento, cero funciones perdidas.

## Después del puerto 1:1

Con la base en componentes, el resto del roadmap avanza más rápido — no antes de eso:

- **Rediseño visual** — la nueva UI (pestañas Hoy/Módulos/Métricas/Perfil, sistema de diseño) se construye sobre esta base, no sobre HTML plano.
- **Motor de primitivas** — checklist, contador, métricas, tarea con fecha, registro y nota, ahora sí como componentes reutilizables de verdad.
- **IA embebida y pagos** — API routes de Next.js o Edge Functions de Supabase (cualquiera funciona igual de bien); TypeScript ayuda justo aquí.